Esercitazione 12 – Assegnata come lavoro autonomo

Per risolvere gli esercizi **scrivete la soluzione di ogni esercizio nel file .py relativo**, che trovate nella cartella dell'esercitazione (ad esempio, per l'esercizio 1 scrivete il vostro programma nel file Ex1.py, per l'esercizio 2, nel file Ex2.py, e così via). Notate che ogni file incorpora del codice python per eseguire alcuni test sulla funzione. NON modificate questo codice, ma **SCRIVETE SOLO il contenuto della funzione**. Per farlo usare l'ambiente Spyder.

In ogni esercizio, se non diversamente richiesto, potete sempre assumere che gli input forniti siano coerenti con la traccia.

Esercizi

- **Ex1(t1,t2)** Si consideri la classe `Tempo` definita nel file `Tempo.py`, il cui costruttore prende in input, nell'ordine, tre numeri interi `o`, `m`, `s` e costruisce un oggetto avente gli attributi `Ore=o`, `Minuti=m` e `Secondi=s`. I tre parametri del costruttore ora descritti sono opzionali ed il valore di default per tutti e tre è pari a `0`. Nella classe `Tempo` sono inoltre definiti i metodi per confrontare due oggetti di tipo `Tempo` con gli operatori ==, `!=`, >, >=, <, <=. Il comportamento di questi operatori è quello ovvio. Ad esempio, siano t1 e t2 due oggetti di tipo `Tempo`, t1>t2 se t1.Ore>t2.Ore, oppure se t1.Ore==t2.Ore e t1.Minuti>t2.Minuti, oppure se t1.Ore==t2.Ore e t1.Minuti==t2.Minuti e t1.Secondi>t2.Secondi.

  Scrivere una funzione che riceve in input due oggetti **t1** e **t2** di tipo Tempo e restituisce la distanza in secondi fra i due tempi. Ad esempio, se **t1** rappresenta l'orario 11:10:00 e **t2** rappresenta l'orario 9:30:15, allora la funzione deve restituire `5985`. Notate che lo stesso risultato è restituito se i due tempi sono invertiti, cioè se **t1** rappresenta l'orario 9:30:15 e **t2** rappresenta l'orario 11:10:00. In altri termini, la funzione deve restituire il valore assoluto della differenza in secondi fra i due tempi.

- **Ex2(l)** Facendo sempre riferimento alla classe `Tempo` descritta nell'esercizio 1, scrivere una funzione che riceve in input una lista **l** contenente oggetti di tipo `Tempo` e restituisce la lista contenente gli stessi oggetti ordinati in modo crescente e senza ripetizioni. Ad esempio, se **l**=`[Tempo(10,15),Tempo(9,12,34),Tempo(8,30,2)]`, la funzione deve restituire `[Tempo(8,30,2),Tempo(9,12,34),Tempo(10,15)]`.

  NOTA: Per la classe `Tempo` è stato ridefinito il metodo `__str__` in modo che la rappresentazione dell'oggetto come stringa sia nella forma *ore:minuti:secondi*. Ad esempio, se stampate l'oggetto `Tempo(8,30,2)` vedrete a schermo `8:30:2`. Questo significa che quando eseguite i test di questo esercizio leggerete a schermo i tempi stampati nel precedente formato. Ad esempio, nel primo test leggerete
  Output atteso:
  [8:30:2, 9:12:34, 10:15:0]
  ma quello che la funzione deve restituire è `[Tempo(8,30,2),Tempo(9,12,34), Tempo(10,15)]`, come indicato in precedenza (si confrontino anche i test nel file dell'esercizio). Questa osservazione vale anche per gli esercizi successivi, ogni volta che la funzione restituisce oggetti (o liste, insiemi o dizionari contenenti oggetti).

- **Ex3(t,s)** Facendo sempre riferimento alla classe `Tempo` descritta nell'esercizio 1, scrivere una funzione che riceve in input un oggetto **t** di tipo `Tempo`, ed un intero **s** e restituisce un oggetto di tipo `Tempo` ottenuto incrementando **t** di **s** secondi. La funzione deve eseguire un incremento "modulo 24", cioè se l'orario ottenuto supera le 23:59:59, la funzione deve restituire un orario del giorno successivo. Potete assumere che non si chieda mai un incremento superiore alle 24 ore. Ad esempio, se **t** rappresenta l'orario 23:30:40 e **s**=2000, la funzione deve restituire un oggetto di tipo Tempo che rappresenta l'orario 00:03:00.

- **Ex4(l)** Si consideri la classe `Persona` definita nel file `Persona.py`, il cui costruttore prende in input, nell'ordine, una stringa `n` ed un intero `a` e costruisce un oggetto avente gli attributi `nome=n` e `annoNascita=a`. Nella classe `Persona` sono inoltre definiti i metodi per confrontare due oggetti di tipo `Persona` con gli operatori == e `!=`. In particolare, due oggetti di tipo `Persona` sono uguali se hanno lo stesso nome, altrimenti sono diversi.

  Scrivere una funzione che riceve in input una lista **l** di oggetti di tipo `Persona` e restituisce un dizionario le cui chiavi sono anni e i cui valori sono insiemi di nomi di persone contenute in **l** nate in quell'anno. Ad esempio, se **l**=`[Persona('mario',1952),Persona('anna',1952),Persona('aldo',1970)]`, la funzione deve restituire `{1952:{'mario','anna'},1970:{'aldo'}}`. Potete assumere che nella lista **l** in input non ci siano persone con lo stesso nome. Notate che le chiavi del dizionario devono essere valorizzate solo ad interi che corrispondono ad anni di nascita delle persone in **l**.

- **Ex5(file)** Facendo sempre riferimento alla classe `Persona` descritta nell'esercizio 4, scrivere una funzione che riceve in input il nome **file** di un file csv contenente righe nel seguente formato

  ```
  Nome,AnnoNascita
  ```

  e che si riferiscono al nome ed all'anno di nascita di una persona e restituisce una lista di persone (oggetti di tipo `Persona`) tale che ciascuna persona nel risultato corrisponda ad una persona rappresentata in una riga nel file in input. Gli elementi nella lista restituita devono essere ordinati in base all'*ordine alfabetico* dei nomi delle persone. *Potete assumere che nel file non ci siano due persone aventi lo stesso nome*. Se il file in input è vuoto, la funzione deve restituire una lista vuota. Ad esempio, se **file** contiene il seguente testo:

  ```
  Nome,AnnoNascita
  Mario,1972
  Aurora,1988
  Elena,1998
  Sara,2003
  ```

  allora la funzione deve restituire la lista: `[Persona('Aurora',1972),Persona('Elena',1998),Persona('Mario',1972),Persona('Sara',2003)]`.

- **Ex6(file)** Si consideri la classe `Squadra` definita nel file `Squadra.py`, il cui costruttore prende in input una stringa `n` e costruisce un oggetto avente l'attributo `nome=n`. Nella classe `Squadra` sono inoltre definiti i metodi per confrontare due oggetti di tipo `Squadra` con gli operatori == e `!=`. In particolare, due oggetti di tipo `Squadra` sono uguali se hanno lo stesso nome, altrimenti sono diversi. Si consideri inoltre la classe `Partita`, il cui costruttore prende in input, nell'ordine, una squadra `s1`, una squadra `s2`, un intero `g1` ed un intero `g2` e costruisce oggetti di tipo `Squadra` aventi gli attributi `casa=s1`, `ospite=s2`, `goalCasa=g1`, `goalOspiti=g2`.

  Scrivere una funzione che prende in input il nome **file** di un file csv contenente righe nel formato

  ```
  Casa,Ospiti,GoalCasa,GoalOspiti
  ```

  e restituisce una lista di oggetti di tipo `Partita`, ognuno corrispondente ad una riga del file, ed i cui elementi sono ordinati nell'ordine in cui le partite compaiono nel file. Ovviamente se il file in input è vuoto o non contiene altre righe oltre la prima, la funzione deve restituire una lista vuota. Ad esempio, se **file** contiene il seguente testo:

  ```
  Casa,Ospiti,GoalCasa,GoalOspiti
  Argentina,Francia,7,5
  Croazia,Marocco,2,1
  Francia,Marocco,2,0
  Argentina,Croazia,3,0
  ```

allora la funzione deve restituire la lista: `[Partita(Squadra('Argentina'), Squadra('Francia'),7,5),Partita(Squadra('Croazia'), Squadra('Marocco'),2,1),Partita(Squadra('Francia'), Squadra('Marocco'),2,0),Partita(Squadra('Argentina'), Squadra('Croazia'),3,0)]`.

- **Ex7(file)** Si consideri nuovamente la classe `Squadra` definita nel file `Squadra.py`, descritta nell'esercizio precedente. Scrivere una funzione che prende in input il nome **file** di un file csv contenente righe nel medesimo formato utilizzato nell'esercizio precedente e restituisce l'insieme delle squadre che sono prime in classifica sulla base dei risultati delle partite contenute in **file.** Ad esempio, se la funzione riceve in input lo stesso file utilizzato come esempio nell'esercizio precedente, la funzione deve restituire `{Squadra('Argentina')}`.

  NOTA: Nella classe `Squadra` è stato definito il metodo `__hash__` . Questo metodo associa ad ogni oggetto della classe un intero rispettando la seguente regola: Due oggetti che sono uguali devono avere anche lo stesso valore hash, ma il contrario non è necessariamente vero. La presenza del metodo `__hash__` in una classe rende gli oggetti di questa classe **immutabili** (in questo specifico caso non è, ad esempio, possibile modificare il nome della squadra), e consente di inserire gli oggetti in un insieme, dato che in Python gli insiemi possono contenere solo oggetti che sono associati ad un valore hash. Inoltre consente anche di definire dizionari in cui oggetti di classe `Squadra` siano chiavi, perché, come noto, i dizionari possono usare come chiavi solo oggetti immutabili (e che hanno un hash associato).